МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Отчет о выполнении лабораторной работы №1 по дисциплине «Архитектура суперкомпьютеров»

Представление дробных чисел в формате bf16

Студент группы 5130201/40101 _____ Гасов А.Е.

Преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

1 Постановка задачи

1. Реализовать консольную программу с меню, предоставляющим пользователю выбор направления перевода: из десятичной системы счисления в двоичную и обратно.

2. Реализовать перевод вещественных чисел в формат bf16. Реализовать кодирование и декодирование, воспроизводя всю теорию представления чисел с плавающей точкой, не полагаясь на готовые библиотечные функции перевода.

2 Математическое описание

bf16 - формат хранения вещественного числа в 16 битах: 1 бит знака s, 8 бит смещённой экспоненты E (смещение 127) и 7 бит мантиссы M. Число хранится в нормализованном виде $1.M \cdot 2^{E-127}$, единица перед точкой не хранится явно. Вещественное число вычисляется по формуле:

$$x = (-1)^s \cdot (1 + M) \cdot 2^{E-127}.$$

Крайние значения экспоненты зарезервированы: $E = 0$ при $M = 0$ даёт ноль, иначе - денормализованное число $x = (-1)^s \cdot M \cdot 2^{1-127}$; $E = 255$ даёт $\pm\infty$ (при $M = 0$) или NaN (при $M \neq 0$).

Перевод из десятичной формы в bf16. Целая часть числа переводится в двоичный вид делением на 2, дробная - умножением на 2 (на каждом шаге дробная часть умножается на 2, целая часть результата - очередной бит, остаток идёт в следующий шаг). Полученная запись нормализуется к виду $1.xxxx \cdot 2^E$ сдвигом до первой единицы, после чего 7 бит мантиссы округляются по следующему за ними биту (округление к ближайшему), а E смещается на 127.

Перевод из bf16 в десятичную форму. Обратная операция: 16 бит разбираются на знак, экспоненту и мантиссу, после чего значение восстанавливается по формуле выше (с учётом особых случаев нуля, денормализованных чисел, бесконечности и NaN).

Перевод целых чисел (без дробной части) выполняется классическим способом - делением на 2 с накоплением остатков (в прямом направлении) и суммированием степеней двойки (в обратном).

3 Особенности реализации

Программа реализована на языке Python и разделена на два модуля: main.py отвечает за консольное меню и перевод целых чисел, а bf16.py - за кодирование и декодирование вещественных чисел в формате bf16. Ниже описаны функции модуля bf16.py.

Функция decimal_to_bf16
Вход: float value - вещественное десятичное число
Выход: строка из 16 бит - представление числа в формате bf16
Описание: Число разбивается на знак, целую и дробную части, каждая часть переводится в двоичный вид (unsigned_integer_to_binary, fractional_part_to_binary). Результат нормализуется к виду $1.mantissa \cdot 2^{exponent}$ (normalize), мантисса округляется до 7 бит (round_mantissa), а экспонента смещается на 127. Отдельно обрабатываются переполнение (число кодируется как $\pm\infty$) и антипереполнение (число кодируется как 0).

```
def decimal_to_bf16(value: float) -> str:
    if value == 0:
        return "0" * TOTAL_BITS

    sign_bit = "1" if value < 0 else "0"
    module = abs(value)

    integer_part = int(module)
    fractional_part = module - integer_part

    integer_bits = unsigned_integer_to_binary(integer_part)
    fractional_bits = fractional_part_to_binary(fractional_part)
    exponent, mantissa_source = normalize(integer_bits, fractional_bits)
    if exponent is None:
        return sign_bit + "0" * (EXPONENT_BITS + MANTISSA_BITS)

    mantissa_bits, exponent_bump = round_mantissa(mantissa_source, MANTISSA_BITS)
    biased_exponent = exponent + exponent_bump + BIAS

    if biased_exponent >= 255:
        return sign_bit + "1" * EXPONENT_BITS + "0" * MANTISSA_BITS
    if biased_exponent <= 0:
        return sign_bit + "0" * (EXPONENT_BITS + MANTISSA_BITS)

    exponent_bits = format(biased_exponent, f"0{EXPONENT_BITS}b")
    return sign_bit + exponent_bits + mantissa_bits
```

Листинг 1: decimal_to_bf16

Функция bf16_to_decimal
Вход: str bits - строка из 16 бит в формате bf16
Выход: float - восстановленное десятичное число
Описание: Строка разбирается на знак, экспоненту и мантиссу. Мантисса переводится обратно в дробь (bits_to_fraction), после чего в зависимости от значения экспоненты выбирается один из трёх случаев: экспонента 0 даёт ноль или денормализованное число, экспонента 255 даёт $\pm\infty$ или NaN, иначе значение вычисляется по обычной формуле $(-1)^{sign}(1 + mantissa) \cdot 2^{exponent-127}$.

```
def bf16_to_decimal(bits: str) -> float:
    bits = bits.strip()
    if len(bits) != TOTAL_BITS or any(bit not in "01" for bit in bits):
        raise ValueError(f"нужно␣ровно␣{TOTAL_BITS}␣бит,␣только␣из␣0␣и␣1")

    sign = -1 if bits[0] == "1" else 1
    exponent_bits = bits[1:1 + EXPONENT_BITS]
```

```
    mantissa_bits = bits[1 + EXPONENT_BITS:]
    biased_exponent = int(exponent_bits, 2)
    mantissa_fraction = bits_to_fraction(mantissa_bits)

    if biased_exponent == 0:
        if mantissa_fraction == 0:
            return sign * 0.0
        return sign * mantissa_fraction * 2 ** (1 - BIAS)

    if biased_exponent == 255:
        return sign * float("inf") if mantissa_fraction == 0 else float("nan")

    return sign * (1 + mantissa_fraction) * 2 ** (biased_exponent - BIAS)
```

Листинг 2: bf16_to_decimal

Функция describe_bf16

Вход: str bits - строка из 16 бит в формате bf16

Выход: строка с разбором знака, экспоненты и мантиссы

Описание: Вспомогательная функция для вывода на экран: разбирает 16 бит на составляющие и печатает знак, смещённую и истинную экспоненту, а также мантиссу и её дробное значение - используется программой для наглядной демонстрации перевода.

```
def describe_bf16(bits: str) -> str:
    sign_bit = bits[0]
    exponent_bits = bits[1:1 + EXPONENT_BITS]
    mantissa_bits = bits[1 + EXPONENT_BITS:]
    biased_exponent = int(exponent_bits, 2)
    return (
        f"знак = {sign_bit} отрицательное({'' if sign_bit == '1' else положительное''})\
    n"
        f"экспонента = {exponent_bits} -> смещённая {biased_exponent}, "
        f"истинная {biased_exponent - BIAS} смещённая( - {BIAS})\n"
        f"мантисса = {mantissa_bits} -> дробная часть {bits_to_fraction(mantissa_bits)}"
    )
```

Листинг 3: describe_bf16

Функция unsigned_integer_to_binary

Вход: int value - неотрицательное целое число

Выход: str - двоичная строка

Описание: Классический перевод целого числа в двоичную систему делением на 2: на каждом шаге остаток от деления на 2 - очередной бит (дописывается слева), а число заменяется на целую часть от деления на 2.

```
def unsigned_integer_to_binary(value: int) -> str:
    if value == 0:
        return "0"
    binary_text = ""
    while value > 0:
        binary_text = str(value % 2) + binary_text
        value //= 2
    return binary_text
```

Листинг 4: unsigned_integer_to_binary

Функция fractional_part_to_binary

Вход: float fractional_part - дробная часть числа, $0 \leqslant x < 1$

Выход: str - двоичные разряды дробной части

Описание: Классический перевод дробной части умножением на 2: на каждом шаге дробная часть умножается на 2, целая часть результата - очередной бит, а остаток идёт

в следующий шаг. Цикл останавливается, когда дробная часть становится точным нулём (любое число типа float - точная двоично-рациональная дробь, поэтому процесс гарантированно завершается), либо когда пройдено MAX_USEFUL_FRACTIONAL_BITS шагов - дальше первую единицу мантиссы искать бессмысленно, так как число в любом случае будет занулено в bf16 из-за нехватки диапазона экспоненты.

```
def fractional_part_to_binary(fractional_part: float) -> str:
    bits = []
    for _ in range(MAX_USEFUL_FRACTIONAL_BITS):
        if fractional_part == 0:
            break
        fractional_part *= 2
        bit = int(fractional_part)
        bits.append(str(bit))
        fractional_part -= bit
    return "".join(bits)
```

Листинг 5: fractional_part_to_binary

Функция normalize

Вход: str integer_bits, fractional_bits - двоичные представления целой и дробной частей

Выход: exponent - истинная экспонента (или None, если число слишком мало для bf16); mantissa_source - биты после подразумеваемой единицы

Описание: Если целая часть не равна нулю, число нормализуется сдвигом до старшего бита целой части (он и есть подразумеваемая единица), а экспонента равна его позиции. Если же целая часть нулевая, ищется первая единица в дробной части - она становится подразумеваемой единицей, а экспонента - отрицательным номером её позиции. Если единица не найдена вовсе, число слишком мало для представления в bf16.

```
def normalize(integer_bits: str, fractional_bits: str):
    if integer_bits != "0":
        exponent = len(integer_bits) - 1
        mantissa_source = integer_bits[1:] + fractional_bits
        return exponent, mantissa_source

    first_one_index = fractional_bits.find("1")
    if first_one_index == -1:
        return None, ""
    exponent = -(first_one_index + 1)
    mantissa_source = fractional_bits[first_one_index + 1:]
    return exponent, mantissa_source
```

Листинг 6: normalize

Функция round_mantissa

Вход: str bits - биты мантиссы; int keep - сколько бит нужно оставить (7 для bf16)

Выход: округлённые биты мантиссы; флаг переноса в экспоненту (0 или 1)

Описание: Из битов мантиссы оставляются первые keep бит, а следующий за ними бит используется для округления к ближайшему: если он 0, лишние биты просто отбрасываются, если 1 - к оставленным битам прибавляется единица. Если при этом мантисса переполняется (была вида $111\ldots1$), она обнуляется, а наружу возвращается флаг переноса, сигнализирующий о необходимости увеличить экспоненту на 1.

```
def round_mantissa(bits: str, keep: int):
    bits = bits.ljust(keep + 1, "0")
    kept_bits, round_bit = bits[:keep], bits[keep]
    if round_bit == "0":
        return kept_bits, 0

    incremented = bin(int(kept_bits, 2) + 1)[2:].zfill(keep)
```

```
    if len(incremented) > keep:
        return "0" * keep, 1
    return incremented, 0
```

Листинг 7: round_mantissa

Функция bits_to_fraction

Вход: str bits - биты мантиссы

Выход: float - дробное значение мантиссы

Описание: Обратная операция к переводу дробной части в биты: значение вычисляется как сумма $\sum_i bit_i \cdot 2^{-(i+1)}$ по всем битам мантиссы, то есть каждый бит на позиции i (считая с 0) даёт вклад $2^{-(i+1)}$.

```
def bits_to_fraction(bits: str) -> float:
    return sum(int(bit) * 2 ** -(position + 1) for position, bit in enumerate(bits))
```

Листинг 8: bits_to_fraction

4 Результаты работы

Перевод вещественного числа -123.45 в формат bf16 и обратно, с разбором результата по битам представлен в Листинге 9. Так как мантисса bf16 ограничена 7 битами, после округления восстановленное число немного отличается от исходного (-123.5 вместо -123.45).

```
Выберите
 направление перевода:
  1. десятичное -> двоичное
  2. двоичное -> десятичное
  0. выход
> 1Тип
 числа:
  1. целое
  2. вещественное (bf16)
> 2Введите
 вещественное десятичное число: -123.45Двоичный
 результат (bf16, 16 бит): 1100001011110111знак
 = 1 отрицательное()экспонента
 = 10000101 -> смещённая 133, истинная 6 смещённая( - 127)мантисса
 = 1110111 -> дробная часть 0.9296875Выберите

 направление перевода:
  1. десятичное -> двоичное
  2. двоичное -> десятичное
  0. выход
> 2Тип
 числа:
  1. целое
  2. вещественное (bf16, 16 бит)
> 2Введите
 16 бит числа в формате bf16: 1100001011110111знак
 = 1 отрицательное()экспонента
 = 10000101 -> смещённая 133, истинная 6 смещённая( - 127)мантисса
 = 1110111 -> дробная часть 0.9296875Десятичный
 результат: -123.5
```

Листинг 9: Перевод вещественного числа -123.45 в bf16 туда и обратно

Заключение

В ходе выполнения лабораторной работы реализовано консольное приложение, конвертирующее чисела из десятичной системы в представление bf16 (и наоборот).

Реализовано:

1. Перевод вещественных чисел из десятичной системы в формат bf16 и обратно: нормализация числа, смещение экспоненты, округление мантиссы до 7 бит, а также обработка нуля, денормализованных чисел, бесконечности и NaN.
2. Консольное меню с выбором направления перевода (десятичное $\leftrightarrow$ двоичное) и типа числа (целое / вещественное).

Список литературы

[1] RISC-V International. 24.1. «BF16» Extensions for BFloat16-precision Floating-Point, Version 1.0 [Электронный ресурс]. URL: https://docs.riscv.org/reference/isa/v20260120/unpriv/bfloat16.html (дата обращения: 17.09.2026).

[2] Хабр: Chekashkin A. FP32, FP16, BF16 и FP8 - разбираемся в основных типах чисел с плавающей запятой [Электронный ресурс] URL: https://habr.com/ru/companies/serverflow/articles/847068/ (дата обращения: 17.09.2026).